3. Solution

The site keeps the three-layer shape from Assessment 2 and adds a second client to it. Nothing in the browser ever talks to MySQL; everything goes through the API.

Table 1 - The four parts of the system.

Layer	Technology	Responsibility
Database	MySQL 8.0	charityevents_db: organisations, categories, events, registrations. Owns the integrity rules (foreign keys, unique key, CHECK constraints).
Server	Node.js 20+ / Express 4.21 / mysql2 3.11	The only process with database credentials. Validates input, runs parameterised SQL, maps results to JSON, decides status codes, and proxies the weather API.
Public client	Angular 22.2 / TypeScript 6	Home, Search, Event details and Register pages. Calls the API with HttpClient and renders the result with signals.
Staff client	Angular 22.2 / TypeScript 6	Event list, new event, edit event. Same API, different endpoints (it may also write).

A page request always follows the same path. The Angular router matches a URL and creates a component; the component asks a service for data; the service calls the API with HttpClient and returns a Promise; Express receives the request, validates the parameters and runs one parameterised query through the mysql2 connection pool; MySQL returns rows; Express serialises them to JSON; the service resolves the Promise; the component writes the result into a signal; and the template, which is already bound to that signal, re-renders. No part of that chain knows about the layer after it, which is why the same API can serve two different front ends and why we could replace the A2 front end without touching a single SQL statement.

Concretely, a booking works like this. On /event/5 the visitor presses Register. The button links to /register?eventId=5 - a query string, not a JavaScript variable, so the page can be refreshed, bookmarked or shared and still knows which event it is for. The register page reads eventId from the URL, fetches the event so it can show the name, date, venue and price, and fills in the ticket total live as the visitor changes the quantity. On submit it POSTs to /api/registrations. The API validates the body, checks that the event exists and is still accepting bookings, inserts the row, and answers 201 with the new record. The page shows a confirmation and a link back to the event, and the event page now lists the new booking first because it sorts by registration date descending.